

Università degli studi di Modena e Reggio Emilia

Dipartimento di Ingegneria Enzo Ferrari

Post Quantum Cryptography

Indice

1	Introduzione	1
2	Lattice-Based Cryptography	4
2.1	Prerequisiti Matematici	4

Capitolo 1

Introduzione

Prima di poter parlare di **Algoritmi Crittografici *Post Quantum***, è necessario andare a capire per quale motivo si sono resi necessari.

Il ***Quantum Computing*** è stato ideato da Yuri Manin e Richard Feynman, che presupponeva di utilizzare i fenomeni della **meccanica quantistica** come la ***superposition***, ***interference*** e ***entanglement*** per eseguire operazioni sui dati.

- un **qubit** è l'analogo quantistico del classico bit, e può essere due valori contemporaneamente con una probabilità diversa per ognuno.
- un ***n-qubit register*** può essere in 2^n stati nello stesso momento, ognuno, anche in questo caso, con la stessa probabilità.

Quando una funzione $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$ è applicato ad un *n-qubit register*, vengono simultaneamente valutati tutti i 2^n stati. Nel momento in cui viene **misurato** il valore di un *n-qubit register*, questo collasserà su uno dei 2^n possibili valori secondo la sua distribuzione di probabilità sottostante. In linea di massima possiamo definire i computer quantistici come “*massively parallel machines*”.

1. La minaccia dei *Quantum Computer*: ***Shor's Algorithm***[4]. L'intero sistema **PKI** utilizza alla sua base tre algoritmi:

1. **RSA**: la cui sicurezza è basata sulla difficoltà della fattorizzazione di numeri interi.
2. **DH**: la cui sicurezza è basata sulla complessità del *discrete logarithm problem*.
3. **ECC**: la cui sicurezza è basata sulla complessità del *elliptic curve discrete logarithm problem*.

Nel 1994, Peter Shor ha scoperto un algoritmo quantistico efficiente (*polynomial time*) per risolvere questi problemi. In questo modo, tutte le implementazioni RSA, DH e ECC possono completamente risolte dai computer quantistici.

2. La minaccia dei *Quantum Computer*: **Grover**[2].

Consideriamo una funzione $F : \{0, 1\}^n \rightarrow \{0, 1\}$ tale che:

- F è facilmente calcolabile
- $F(x) = 1$ per esattamente un input $x \in \{0, 1\}^n$

Grover's algorithm: nel 1996, Lov Grover ha scoperto un algoritmo quantistico che permettesse di trovare la $x \in \{0, 1\}^n \mid F(x) = 1$ in $\sqrt{2^n}$ valutazione della funzione F .

Exhaustive key search: se consideriamo AES con una chiave (*secret key*) lunga ℓ -bit. Supponiamo di conoscere t ovvero una coppia plaintext-ciphertext (m_i, c_i) , dove t è tale che il numero di falsi positivi – è una chiave k' che permette di risolvere $c_i = \text{AES}_{k'}(m_i)$ con $k' \neq k$ – sulle chiavi è molto vicino a 0.

Possiamo definire $F : \{0, 1\}^\ell \rightarrow \{0, 1\}$ dove $F(k) = 1 \Leftrightarrow \text{AES}_k(m_i) = c_i \forall i \in \{1, t\}$ e $F(k) = 0$ negli altri casi. Allora l'algoritmo di Grover può trovare la chiave segreta non in 2^ℓ operazioni classiche, ma in $2^{\ell/2}$ **operazioni quantistiche**.

Timeline dei Quantum Computer

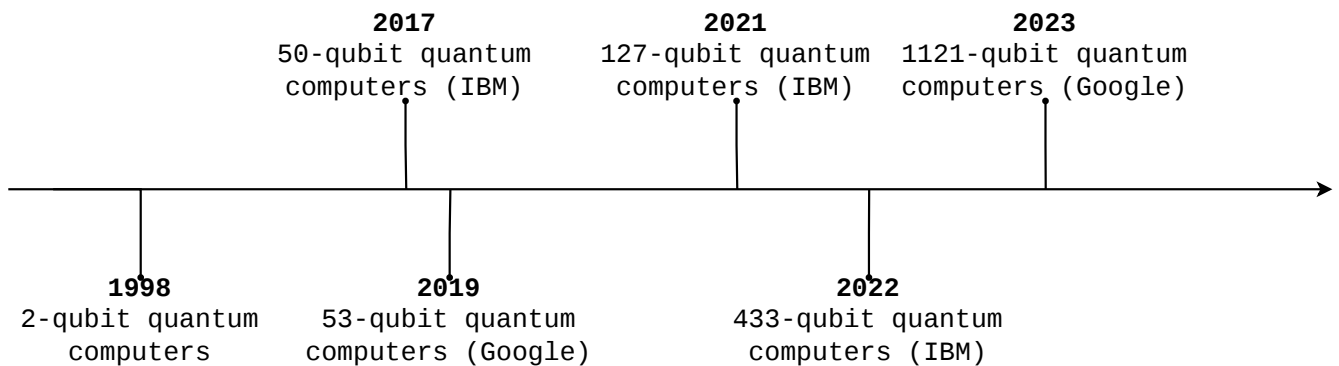


Figura 1.1: Evoluzione del numero dei qubits nei Quantum Computer

Per fattorizzare una chiave RSA a 2048 bit in tempo polinomiale sfruttando l'algoritmo di Shor, un computer quantistico richiede registri composti da diverse migliaia di qubit — almeno registri a 2048 qubits. Attenzione, però, non tutti i qubit sono uguali: c'è una grande differenza tra **qubit fisici** (l'hardware reale) e **qubit logici** (l'unità di calcolo vera e propria). Poiché i qubit fisici sono estremamente delicati e inclini a generare errori per via del rumore ambientale e della decoerenza, non è possibile utilizzarli direttamente per calcoli complessi. Per renderli ***fault-tolerant*** è necessario applicare il protocollo ***Quantum Error Correction (QEC)***[3], il quale permette di raggruppare un elevato numero di qubit fisici — singolarmente instabili e imperfetti — per sintetizzare un unico qubit logico, stabile e affidabile per il calcolo.

Viene stimato che siano necessari migliaia di qubit fisici per costruire un qubit logico. In questo modo per riuscire a costruire un registro di almeno 2048 qubit logici sono necessari milioni di qubit fisici. Uno studio del 2021, portato avanti da Gidney e Ekers [1] stima che siano necessari 6,000 qubit logici e 20,000,000 qubit fisici per riuscire a fattorizzare una chiave RSA in 8 ore.

La minaccia principale, che però non è così immediata, degli algoritmi di Shor e Grover è la possibilità di catturare e memorizzare grosse quantità di traffico internet fino a che non ci sarà la possibilità di decifrarlo senza la conoscenza della chiave privata: ***Harvest Now Decrypt Later*** (HNDL) *attacks*.

Nome Algoritmo	Tipologia	KEM o DSA	Standard
Kyber	Lattice-based	KEM	FIPS 203
Dilithium	Lattice-based	DSA	FIPS 204
SPHINCS+	Hash-based	DSA	FIPS 205
Falcon	Lattice-based	DSA	FIPS 206 (Draft / In arrivo)
LMS	Hash-based	DSA	SP 800-208
XMSS	Hash-based	DSA	SP 800-208

Tabella 1.1: Algoritmi crittografici Post-Quantum standardizzati dal NIST.

Capitolo 2

Lattice-Based Cryptography

2.1 Prerequisiti Matematici

Aritmetica Modulare

Consideriamo un intero $q \in \mathbb{N} \mid q \geq 2$. Definiamo $a \equiv b \pmod{q}$ significa che $\exists k \in \mathbb{Z} \mid k \cdot q = a - b$, quindi vuol dire che $a - b$ è un **multiplo** di q . Andiamo a definire r come il resto della divisione intera tra $a \in \mathbb{Z}$ e $q \in \mathbb{N}$ e si scrive $r = a \pmod{q}$ che può essere riscritto tramite la definizione della **divisione euclidea** $a = q \cdot k + r$ quindi $0 \leq r < q$.

Definiamo l'insieme $\mathbb{Z}_q = \{0, 1, 2, \dots, q-1\}$ dove le operazioni di **addizione**, **sottrazione** e **moltiplicazione** vengono eseguite $\pmod{q}$.

Anelli Polinomiali (*Polynomial Rings*)

Consideriamo $q \in \mathbb{N}$ un modulo primo. Definiamo $\mathbb{Z}_q[x]$ il set di tutti i polinomi in x con coefficienti in $\mathbb{Z}_q$. Quando viene eseguita operazioni — addizione, sottrazioni, moltiplicazione e divisioni — polinomiali in $\mathbb{Z}_q[x]$ tutti i coefficienti aritmetici sono eseguiti in $\mathbb{Z}_q$.

Esempio

Consideriamo $q = 7$ e consideriamo $f(x) = 5 + 4x^2 + 3x^3 \in \mathbb{Z}_7[x]$ e $g(x) = 6 + 3x + 2x^2 \in \mathbb{Z}_7[x]$.

$$f(x) + g(x) = 4 + 3x + 6x^2 + 3x^3$$

$$f(x) - g(x) = 6 + 4x + 2x^2 + 3x^3$$

$$f(x) \times g(x) = 2 + x + 6x^2 + 2x^3 + 3x^4 + 6x^5$$

L'Anello Polinomiale $R_q = \mathbb{Z}_q[x]/(x^n + 1)$

Consideriamo $q \in \mathbb{N}$ un modulo primo e $n \in \mathbb{N}$. Il **Polynomial Ring** $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ è composto dai polinomi in $\mathbb{Z}_q[x]$ di grado minore di n , con le moltiplicazioni dei polinomi eseguito modulo il **polinomio di riduzione (reduction polynomial)** $x^n + 1$ (in parole semplici il risultato della moltiplicazione è il resto della divisione polinomiale modulare). Quindi la moltiplicazione tra $f(x), g(x) \in R_q$:

- calcolare $h(x) = f(x) \cdot g(x)$ $h(x) \in \mathbb{Z}_q$ dove il grado è al massimo $2n - 2$
- dividere $h(x)$ per il polinomio $x^n + 1$ e calcoliamo il resto $r(x)$ con grado al massimo $n - 1$
- l'operazione $f(x) \times g(x) = r(x) \in R_q$

Esempio

Consideriamo $q = 41$ e $n = 4$. Quindi R_q comprende i polinomi in $\mathbb{Z}_{41}[x]$ di grado al massimo 3. Consideriamo $f(x) = 32 + 17x^2 + 22x^3 \in R_q$ e $g(x) = 11 + 7x + 19x^2 + x^3 \in R_q$.

$$h(x) = f(x) \times g(x) = 24 + 19x + 16x^2 + 24x^3 + 26x^4 + 25x^5 + 22x^6$$

A questo punto bisogna eseguire la divisione di $h(x)$ per il polinomio $x^4 + 1$ (è possibile eseguirla andando a sostituire x^4 con -1 , x^5 con $-x$, e x^6 con $-x^2$ e dopo semplificare). In questo modo otteniamo:

$$r(x) = 24 + 19x + 16x^2 + 24x^3 - 26 - 25x - 22x^2 = 39 + 35x + 35x^2 + 24x^3$$

Quindi il risultato dell'operazione $f(x) \times g(x) = 39 + 35x + 35x^2 + 24x^3 \in R_q$

Rappresentazione di un Polinomio come un Vettore

Un polinomio $f(x) = a_0 + a_1x + \dots + a_{n-1}x^{n-1} \in R_q = \mathbb{Z}[x]/(x^n + 1)$ può essere rappresentato come il vettore dei coefficienti $f = (a_0, a_1, \dots, a_{n-1})$; la cui lunghezza è esattamente n .

Esempio

Consideriamo l'anello polinomiale $R_q = \mathbb{Z}_{41}[x]/(x^4 + 1)$. Definiamo su di esso due polinomi: $f(x) = 23 + 11x^2 + 7x^3 \in R_q$ e $g(x) = 40 + 5x + 16x^2 \in R_q$, essi possono essere rappresentati tramite i loro vettori:

$$f = (23, 0, 11, 7)$$

$$g = (40, 5, 16, 0)$$

In R_q avremo quindi:

- $f + g = (23, 0, 11, 7) + (40, 5, 16, 0) = (63, 5, 27, 7) \mod 41 = (22, 5, 27, 7)$
- $f - g = (23, 0, 11, 7) - (40, 5, 16, 0) = (-17, -5, -5, 7) \mod 41 = (24, 36, 36, 7)$
- $f \times g = (23, 0, 11, 7) \times (40, 5, 16, 0) = (12, 3, 29, 7)$

Il modulo R_q^k

Consideriamo $k \in \mathbb{N}$. Gli elementi del **modulo** R_q^k sono i vettori di lunghezza k di polinomi in R_q . Le operazioni di addizione e sottrazione degli elementi R_q^k sono **component-wise** e il risultato è anch'esso un elemento in R_q^k . L'**inner product** di due vettori in R_q^k è un polinomio $f(x) \in R_q$. Tutti i vettori in R_q^k sono rappresentati come vettori colonna.

Esempio

Consideriamo $q = 137$, $n = 4$, $R_q = \mathbb{Z}_{137}[x](x^4+1)$ e $k = 3$. Consideriamo due vettori $a, b \in R_q^k$:

$$a = \begin{bmatrix} 93 + 51x + 34x^2 + 54x^3 \\ 27 + 87x + 81x^2 + 6x^3 \\ 112 + 15x + 46x^2 + 122x^3 \end{bmatrix} \quad b = \begin{bmatrix} 40 + 78x + x^2 + 119x^3 \\ 11 + 31x + 57x^2 + 90x^3 \\ 108 + 72x + 47x^2 + 14x^3 \end{bmatrix}$$

$$a + b = \begin{bmatrix} 133 + 129x + 35x^2 + 173x^3 \\ 38 + 118x + 138x^2 + 96x^3 \\ 120 + 87x + 93x^2 + 136x^3 \end{bmatrix} \mod 137 = \begin{bmatrix} 133 + 129x + 35x^2 + 36x^3 \\ 38 + 118x + x^2 + 96x^3 \\ 120 + 87x + 93x^2 + 126x^3 \end{bmatrix}$$

$$\text{e } a^T + b = a[1]b[1] + a[2]b[2] + a[3]b[3] = 93 + 59x + 44x^2 + 132x^3$$

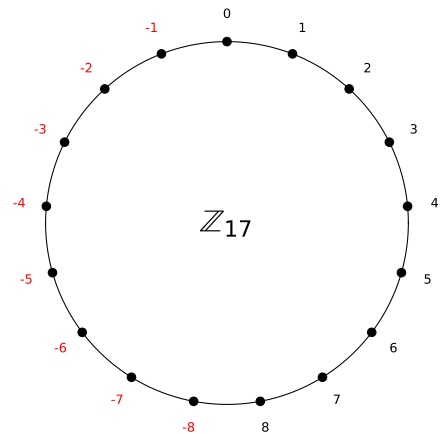
Introduciamo il concetto di **size** sia per gli **interi** in $\mathbb{Z}_q$, che per i **polinomi** in $R_q = \mathbb{Z}_q[x](x^n + 1)$ ed infine per i vettori di polinomi in R_q^k . La **dimensione** viene calcolata come l'**"infinity norm"**, che viene rappresentata con $\|\cdot\|_\infty$ per approfondire queste concetto avremo bisogno prima di definire il concetto di **modulo simmetrico**.

Symmetric mod: q dispari

Consideriamo $q \in \mathbb{N}$ **dispari** e $r \in \mathbb{Z}_q$. Allora valuteremo:

$$r \bmod q = \begin{cases} r & \text{if } r \leq (q-1)/2 \\ r-1 & \text{if } r > (q-1)/2 \end{cases}$$

in questo modo avremo che $-(q-1)/2 \leq r \bmod q \leq (q-1)/2$.

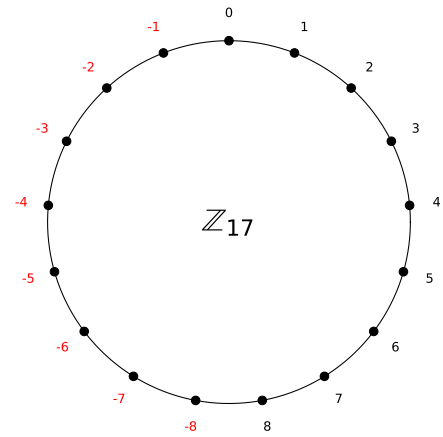


Symmetric mod: q pari

Consideriamo $q \in \mathbb{N}$ **pari** e $r \in \mathbb{Z}_q$. Allora valuteremo:

$$r \bmod q = \begin{cases} r & \text{if } r \leq q/2 \\ r - q & \text{if } r > q/2 \end{cases}$$

in questo modo avremo che $-q/2 < r \bmod q \leq q/2$. Possiamo scrivere sia $\bmod q$ che utilizzare la seguente sintassi: $\bmod^+ q$.



Possiamo scrivere sia $\bmod q$ che utilizzare la seguente sintassi: $\bmod^+ q$.

Bibliografia

- [1] Craig Gidney and Martin Ekerå. How to factor 2048 bit rsa integers in 8 hours using 20 million noisy qubits. *Quantum*, 5:433, April 2021.
- [2] Lov K. Grover. A fast quantum mechanical algorithm for database search, 1996.
- [3] Peter W. Shor. Scheme for reducing decoherence in quantum computer memory. *Phys. Rev. A*, 52:R2493(R)–R2496(R), Oct 1995.
- [4] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, October 1997.